

Compulsory Assignment 3: Semantic Segmentation With U-net

Please fill out the the group name, number, members and optionally the name below.

Group number: 3 **Group member 1:** Sara Marie Åseng Almdahl **Group member 2:** Fredrik Gard Haugen **Group member 3:** Kim Andre Nielsen **Group name (optional):**

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we reccomend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to segment the images using a U-Net. High accuracy is not required, as the purpose of this assignment is to evaluate the correct implementation of the U-Net architecture and workflow.

NOTE: It is recommended to use Google Colab with GPU enabled for CA3 taska. Keras and TensorFlow (version ≥ 2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA3

To work on the CA3 assignment in Google Colab, follow these steps:

1. Connect Google Colab to Google Drive

Your CA3 files should be stored in Google Drive so Colab can access them. Run the following code to mount your Drive:

```
# from google.colab import drive
# drive.mount('/content/drive')
```

After running this, you will be asked to grant access. Once mounted, your Drive will be available under `/content/drive/MyDrive/`.

2. Upload the CA3 Folder

- First, unzip the **CA3** folder on your local computer.
- Then upload the unzipped CA3 folder into your Google Drive (for example inside **MyDrive**).

Example path: `/content/drive/MyDrive/CA3`

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime** → **Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

3. Add CA3 to Python's Search Path

To make sure Python finds the `utilities.py` and other scripts in CA3, add the path (the path is just an example):

```
import sys
```

```
sys.path.append("/content/drive/MyDrive/CA3")
```

Adjust the path above to match the actual location of your CA3 folder in Google Drive.

4. Open the CA3 Notebook

- Upload the CA3 Jupyter notebook (for example, `CA3.ipynb`) to Colab.
 - Once opened, you can run the notebook and it will access the files in your CA3 folder.
-

5. Watch the First Lecture

The first lecture explains how to use Google Colab effectively.
Review it if you are new to Colab.

That's it — you are now ready to run your CA3 code on Google Colab.

Introduction

Chest X-ray(CXR) images are important in medical imaging because it helps focus analysis on the lung region, making disease detection (eg: pneumonia, COVID-19) more accurate and reliable. The purpose is to create clear lung masks that isolate the lungs from surrounding anatomy.

Lung segmentation is the process of separating lung regions from the rest of a chest X-ray. In this assignment, you are provided with CXR images and corresponding binary masks, where pixel values of 0 represent non-lung areas and 1 represents lung tissue. Your task is to focus on semantic segmentation using a U-Net–style architecture. You will segment lung regions from chest X-ray images.

Dataset

The dataset consists of 200 images with corresponding binary masks:

- **Images:** 200 chest X-ray images (grayscale).
- **Masks:** 200 binary masks.
 - 1 = lung
 - 0 = background

Structure

- Images: `CXR_png/`
- Masks: `masks/`
Mask filenames may use suffixes `_mask`.

Pairing rule: For `X.png`, expect `X_mask.png` (accepted suffixes).
You must programmatically verify pairs and report any missing masks.

`image.png`

Assignment Tasks

1. Build a file-pairing script to match each image with its mask; print a summary of matched pairs and missing files.
2. Create train/val/test splits (80/10/10) with a fixed random seed.
3. Implement preprocessing and mask-safe augmentations (flips, crop/zoom; nearest-neighbor resize for masks).

4. Implement the classical U-Net structure that you have learned about in the lectures.
5. Implement a U-Net with Transfer Learning (e.g., ResNet50, VGG16, MobileNetV2).
6. Train your models with a composite loss (e.g., $0.5 \cdot \text{BCE} + 0.5 \cdot \text{DiceLoss}$), track Dice and IoU metrics, and use callbacks (ModelCheckpoint, EarlyStopping).
7. Evaluate on the test set; report metrics and show qualitative results.

Task 0.1 : Data Organization & Pairing

- Build a file-pairing script to match each image with its mask.
- Print a summary of matched pairs and missing files.
- Show the distribution of training, validation and test set

Set DATA_ROOT to the path of the Images folder located in your CA3 assignment folder.

```
import os, glob, random
import numpy as np
import matplotlib.pyplot as plt
import warnings
import matplotlib
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

import importlib
ut = importlib.import_module("utilities")
importlib.reload(ut)

#build_pairs = getattr(ut, "build_pairs")
#report_pairs = getattr(ut, "report_pairs")
#peek_pairs = getattr(ut, "peek_pairs")

DATA_ROOT = r'C:\Users\saraa\OneDrive\Ås\7. semester\DAT300\CA3\
Images'
IMG_DIR = os.path.join(DATA_ROOT, "CXR_png")
MSK_DIR = os.path.join(DATA_ROOT, "masks")

pairs, missing = ut.build_pairs(IMG_DIR, MSK_DIR)
ut.report_pairs(pairs, missing)
ut.peak_pairs(pairs, n_show=3)

train, val, test = ut.split_dataset(pairs, seed=42)
ut.plot_split_distribution(train, val, test)
```

Total images: 200
Matched pairs: 200
Missing masks: 0

Image CHNCXR_0005_0.png



Mask



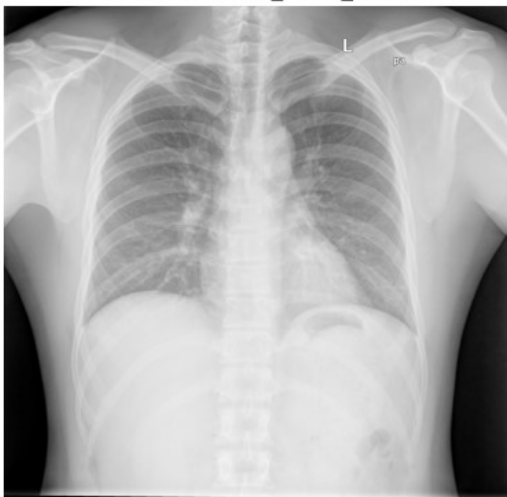
Image CHNCXR_0145_0.png



Mask

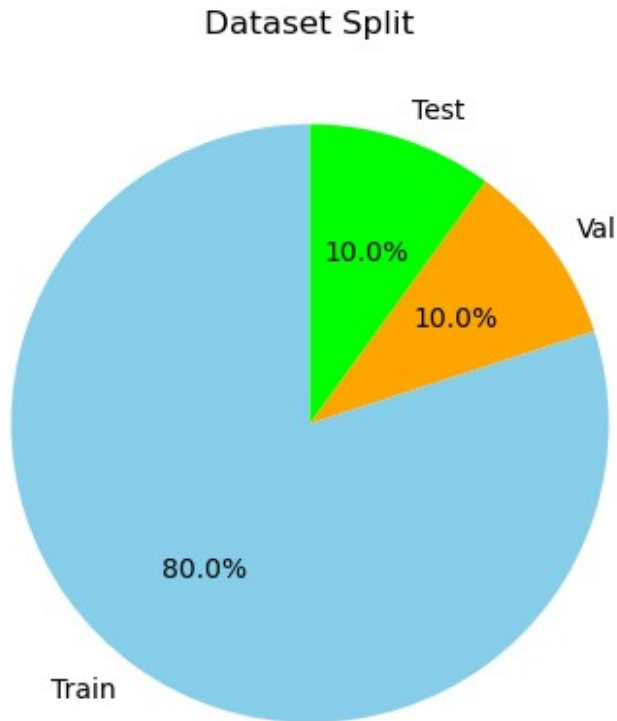


Image CHNCXR_0068_0.png



Mask





Task 0.1 : Preprocessing & Augmentation

- Implement preprocessing for images and masks.
- Ensure masks are not corrupted during transformations.

Image

- Decode as RGB (3 channels), resize to `IMG_SIZE`.
- Use ResNet preprocess.

Mask

- Decode as 1 channel, nearest resize (to preserve labels).
- Binarize.

Augmentations (mask-safe)

- Horizontal flip
- Light zoom/shift via pad + random crop
- Light brightness/contrast jitter (image only)

```
# Build datasets
train_ds = ut.make_dataset(train, augment_data=False) # no
augmentation
val_ds = ut.make_dataset(val, augment_data=False)
```

```

train_aug_ds = ut.make_dataset(train, augment_data=True)

# -----
# Check dataset sizes
# -----
print("Pairs available:")
print("  Train set:", len(train), "pairs")
print("  Val set  :", len(val), "pairs")
print("  Test set :", len(test), "pairs")

# Count actual batches and images
def count_batches_and_images(ds, name):
    n_batches = 0
    n_images = 0
    for imgs, msk in ds:
        n_batches += 1
        n_images += imgs.shape[0]
    print(f"{name}: {n_batches} batches, {n_images} images")

count_batches_and_images(train_ds, "Train dataset (no aug)")
count_batches_and_images(val_ds, "Validation dataset")
count_batches_and_images(train_aug_ds, "Train dataset (with aug)")

# -----
# Inspect shapes before augmentation
# -----
for imgs, msk in train_ds.take(1):
    print("Before augmentation - images:", imgs.shape, "masks:",
msks.shape)

# Inspect shapes after augmentation
for imgs, msk in train_aug_ds.take(1):
    print("After augmentation - images:", imgs.shape, "masks:",
msks.shape)

Pairs available:
  Train set: 160 pairs
  Val set  : 20 pairs
  Test set : 20 pairs
Train dataset (no aug): 40 batches, 160 images
Validation dataset: 5 batches, 20 images
Train dataset (with aug): 40 batches, 160 images
Before augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)
After augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)

```


Part 1 - Model Implementation

Intersection over Union

The IoU score is a popular metric in both segmentation and object detection problems.

If you want to use the `plot_training_history` function in the `utilities.py` library remember to compile the model with the TP, TN, FP, FN metrics such that you can estimate the *Intersection-over-Union*.

However, it is voluntary to estimate IoU.(check the `utilities.py`)

See example below:

```
"""
from tensorflow.keras.metrics import FalseNegatives, FalsePositives,
TrueNegatives, TruePositives

f1_score = getattr(ut, "f1_score")
plot_training_history = getattr(ut, "plot_training_history")

# Example model compile
model.compile(
    optimizer="Something",
    loss="Something else",
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        f1_score,
        # Other metrics of your choice
    ]
)

training_history = model.fit(X_train, y_train, ...)
plot_training_history(training_history)
""";
```

You have also been provided with a custom F1-score metric in the `utilities.py` library, which is specific for image segmentation.

This is mandatory to use when compiling the model.

Task 1.1: Model Implementation

Implement the classical U-net structure that you have learned about in the lectures.
Feel free to experiment with the number of layers, loss-function, batch-normalization, etc.

Remember to compile with the F1-score metric.

```

from tensorflow.keras.metrics import FalseNegatives, FalsePositives,
TrueNegatives, TruePositives, F1Score
from tensorflow.keras.layers import Conv2D, Conv2DTranspose,
BatchNormalization, Activation, MaxPooling2D, Dropout, concatenate
from tensorflow.keras.models import Model
from sklearn.metrics import accuracy_score

def conv2d_block(input_tensor, n_filters, kernel_size = 3, batchnorm =
True):
    """Function to add 2 convolutional layers with the parameters
passed to it"""
    # first layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
kernel_size),\
               kernel_initializer = 'he_normal', padding = 'same')
    (input_tensor)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # second layer
    x = Conv2D(filters = n_filters, kernel_size = (kernel_size,
kernel_size),\
               kernel_initializer = 'he_normal', padding = 'same')(x)
    if batchnorm:
        x = BatchNormalization()(x)
    x = Activation('relu')(x)

    return x

# Code extracted from lecture notes

def get_unet(input_img, n_filters = 16, dropout = 0.1, batchnorm =
True, n_classes = 2):
    # Contracting Path
    c1 = conv2d_block(input_img, n_filters * 1, kernel_size = 3,
batchnorm = batchnorm)
    p1 = MaxPooling2D((2, 2))(c1)
    p1 = Dropout(dropout)(p1)

    c2 = conv2d_block(p1, n_filters * 2, kernel_size = 3, batchnorm =
batchnorm)
    p2 = MaxPooling2D((2, 2))(c2)
    p2 = Dropout(dropout)(p2)

    c3 = conv2d_block(p2, n_filters * 4, kernel_size = 3, batchnorm =
batchnorm)
    p3 = MaxPooling2D((2, 2))(c3)
    p3 = Dropout(dropout)(p3)

```

```

    c4 = conv2d_block(p3, n_filters * 8, kernel_size = 3, batchnorm =
batchnorm)
    p4 = MaxPooling2D((2, 2))(c4)
    p4 = Dropout(dropout)(p4)

    c5 = conv2d_block(p4, n_filters = n_filters * 16, kernel_size = 3,
batchnorm = batchnorm)

    # Expansive Path
    u6 = Conv2DTranspose(n_filters * 8, (3, 3), strides = (2, 2),
padding = 'same')(c5)

    u6 = concatenate([u6, c4])
    u6 = Dropout(dropout)(u6)
    c6 = conv2d_block(u6, n_filters * 8, kernel_size = 3, batchnorm =
batchnorm)

    u7 = Conv2DTranspose(n_filters * 4, (3, 3), strides = (2, 2),
padding = 'same')(c6)
    u7 = concatenate([u7, c3])
    u7 = Dropout(dropout)(u7)
    c7 = conv2d_block(u7, n_filters * 4, kernel_size = 3, batchnorm =
batchnorm)

    u8 = Conv2DTranspose(n_filters * 2, (3, 3), strides = (2, 2),
padding = 'same')(c7)
    u8 = concatenate([u8, c2])
    u8 = Dropout(dropout)(u8)
    c8 = conv2d_block(u8, n_filters * 2, kernel_size = 3, batchnorm =
batchnorm)

    u9 = Conv2DTranspose(n_filters * 1, (3, 3), strides = (2, 2),
padding = 'same')(c8)
    u9 = concatenate([u9, c1])
    u9 = Dropout(dropout)(u9)
    c9 = conv2d_block(u9, n_filters * 1, kernel_size = 3, batchnorm =
batchnorm)

    outputs = Conv2D(n_classes, (1, 1), activation='sigmoid')(c9)
    model = Model(inputs=[input_img], outputs=[outputs])
    return model

```

Task 1.2: Train the model, and plot the training history

Feel free to use the `plot_training_history` function from the provided library `utilities.py`.

```

input_img = keras.Input(shape=(512, 512, 3))
model = get_unet(input_img, n_filters = 8, dropout = 0.0, batchnorm =

```

```

True, n_classes = 1)
model.summary()

# Example model compile
model.compile(
    optimizer="SGD",
    loss=ut.bce_dice_loss,
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        ut.f1_score,
        #accuracy_score,
        ut.iou_soft,
        ut.dice_loss
        # Other metrics of your choice
    ]
)

# training_history = model.fit(train_ds)

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 512, 512, 3)	0	-
conv2d (Conv2D) input_layer[0][0]	(None, 512, 512, 8)	224	
batch_normalization (BatchNormalizatio...	(None, 512, 512, 8)	32	conv2d[0][0]
activation batch_normalizat...	(None, 512, 512, 8)	0	

(Activation)	8)		
conv2d_1 (Conv2D)	(None, 512, 512,	584	activation[0]
[0]	8)		
batch_normalization_1	(None, 512, 512,	32	conv2d_1[0]
[0]	(BatchNormalization)	8)	
activation_1	(None, 512, 512,	0	
batch_normalization_1	(Activation)	8)	
max_pooling2d	(None, 256, 256,	0	
activation_1[0]	(MaxPooling2D)	8)	
dropout	(None, 256, 256,	0	
max_pooling2d[0]	8)		
conv2d_2 (Conv2D)	(None, 256, 256,	1,168	dropout[0][0]
	16)		
batch_normalization_2	(None, 256, 256,	64	conv2d_2[0]
[0]	(BatchNormalization)	16)	
activation_2	(None, 256, 256,	0	
batch_normalization_2	(Activation)	16)	

conv2d_3 (Conv2D)	(None, 256, 256,	2,320		
activation_2[0][...]	16)			
batch_normalization_3	(None, 256, 256,	64	conv2d_3[0]	
[0]	(BatchNormalization)	16)		
activation_3	(None, 256, 256,	0		
batch_normalization_3	(Activation)	16)		
max_pooling2d_1	(None, 128, 128,	0		
activation_3[0][...]	(MaxPooling2D)	16)		
dropout_1 (Dropout)	(None, 128, 128,	0		
max_pooling2d_1[...]	16)			
conv2d_4 (Conv2D)	(None, 128, 128,	4,640	dropout_1[0]	
[0]	32)			
batch_normalization_4	(None, 128, 128,	128	conv2d_4[0]	
[0]	(BatchNormalization)	32)		
activation_4	(None, 128, 128,	0		
batch_normalization_4	(Activation)	32)		

conv2d_5 (Conv2D)	(None, 128, 128,	9,248	
activation_4[0][...	32)		
batch_normalizatio...	(None, 128, 128,	128	conv2d_5[0]
[0]	(BatchNormalizatio...	32)	
activation_5	(None, 128, 128,	0	
batch_normalizat...	(Activation)	32)	
max_pooling2d_2	(None, 64, 64,	0	
activation_5[0][...	(MaxPooling2D)	32)	
dropout_2 (Dropout)	(None, 64, 64,	0	
max_pooling2d_2[...	32)		
conv2d_6 (Conv2D)	(None, 64, 64,	18,496	dropout_2[0]
[0]	64)		
batch_normalizatio...	(None, 64, 64,	256	conv2d_6[0]
[0]	(BatchNormalizatio...	64)	
activation_6	(None, 64, 64,	0	
batch_normalizat...	(Activation)	64)	

conv2d_7 (Conv2D)	(None, 64, 64,	36,928	
activation_6[0][...]	64)		
batch_normalization_7 [0]	(None, 64, 64,	256	conv2d_7[0]
(BatchNormalization)	64)		
activation_7	(None, 64, 64,	0	
batch_normalization_7 [0]	64)		
(Activation)			
max_pooling2d_3	(None, 32, 32,	0	
activation_7[0][...]	64)		
(MaxPooling2D)			
dropout_3 (Dropout)	(None, 32, 32,	0	
max_pooling2d_3[...]	64)		
conv2d_8 (Conv2D)	(None, 32, 32,	73,856	dropout_3[0]
[0]	128)		
batch_normalization_8 [0]	(None, 32, 32,	512	conv2d_8[0]
(BatchNormalization)	128)		
activation_8	(None, 32, 32,	0	
batch_normalization_8 [0]	128)		
(Activation)			

conv2d_9 (Conv2D)	(None, 32, 32,	147,584	
activation_8[0][...]	128)		
batch_normalization_9 [0]	(None, 32, 32,	512	conv2d_9[0]
(BatchNormalization)	128)		
activation_9	(None, 32, 32,	0	
batch_normalization_9 [0]	128)		
(Activation)			
conv2d_transpose	(None, 64, 64,	73,792	
activation_9[0][...]	64)		
(Conv2DTranspose)			
concatenate	(None, 64, 64,	0	
conv2d_transpose...	128)		
(Concatenate)			
activation_7[0][...]			
dropout_4 (Dropout)	(None, 64, 64,	0	
concatenate[0][0]	128)		
conv2d_10 (Conv2D)	(None, 64, 64,	73,792	dropout_4[0]
[0]	64)		
batch_normalization_10 [0]	(None, 64, 64,	256	conv2d_10[0]
(BatchNormalization)	64)		
activation_10	(None, 64, 64,	0	

batch_normalizat...	(Activation)	64)		
conv2d_11 (Conv2D)	(None, 64, 64,	36,928		
activation_10[0]...	64)			
batch_normalizatio...	(None, 64, 64,	256	conv2d_11[0]	
[0]	(BatchNormalizatio...	64)		
activation_11	(None, 64, 64,	0		
batch_normalizat...	(Activation)	64)		
conv2d_transpose_1	(None, 128, 128,	18,464		
activation_11[0]...	(Conv2DTranspose)	32)		
concatenate_1	(None, 128, 128,	0		
conv2d_transpose...	(Concatenate)	64)		
activation_5[0][...				
dropout_5 (Dropout)	(None, 128, 128,	0		
concatenate_1[0]...	64)			
conv2d_12 (Conv2D)	(None, 128, 128,	18,464	dropout_5[0]	
[0]	32)			
batch_normalizatio...	(None, 128, 128,	128	conv2d_12[0]	
[0]				

(BatchNormalization...	32)		
activation_12 batch_normalizat...	(None, 128, 128, 0		
(Activation)	32)		
conv2d_13 (Conv2D) activation_12[0]...	(None, 128, 128, 9,248		
	32)		
batch_normalization...	(None, 128, 128, 128 conv2d_13[0]		
[0]	(BatchNormalization...	32)	
activation_13 batch_normalizat...	(None, 128, 128, 0		
(Activation)	32)		
conv2d_transpose_2 activation_13[0]...	(None, 256, 256, 4,624		
(Conv2DTranspose)	16)		
concatenate_2 conv2d_transpose...	(None, 256, 256, 0		
(Concatenate)	32)		
activation_3[0][...			
dropout_6 (Dropout) concatenate_2[0]...	(None, 256, 256, 0		
	32)		
conv2d_14 (Conv2D) [0]	(None, 256, 256, 4,624 dropout_6[0]		
	16)		

batch_normalization_14[0]	(None, 256, 256, 16)	64	conv2d_14[0]
activation_14	(None, 256, 256, 16)	0	
conv2d_15 (Conv2D)	(None, 256, 256, 16)	2,320	
batch_normalization_15[0]	(None, 256, 256, 16)	64	conv2d_15[0]
activation_15	(None, 256, 256, 16)	0	
conv2d_transpose_3	(None, 512, 512, 8)	1,160	
concatenate_3	(None, 512, 512, 16)	0	
dropout_7 (Dropout)	(None, 512, 512, 16)	0	

conv2d_16 (Conv2D)	(None, 512, 512,	1,160	dropout_7[0]	
[0]	8)			
batch_normalization_16	(None, 512, 512,	32	conv2d_16[0]	
[0]	(BatchNormalization)	8)		
activation_16	(None, 512, 512,	0		
batch_normalization_16	(Activation)	8)		
conv2d_17 (Conv2D)	(None, 512, 512,	584		
activation_16[0]	8)			
batch_normalization_17	(None, 512, 512,	32	conv2d_17[0]	
[0]	(BatchNormalization)	8)		
activation_17	(None, 512, 512,	0		
batch_normalization_17	(Activation)	8)		
conv2d_18 (Conv2D)	(None, 512, 512,	9		
activation_17[0]	1)			

Total params: 543,161 (2.07 MB)

Trainable params: 541,689 (2.07 MB)

Non-trainable params: 1,472 (5.75 KB)

```
#training_history = model.fit(train_ds, validation_data=val_ds)
```

```
history = model.fit(  
    train_aug_ds,  
    validation_data=val_ds,  
    epochs=5,  
    batch_size=40,  
    shuffle=True,  
    verbose=1  
)
```

Epoch 1/5

```
40/40 ————— 34s 589ms/step - dice_loss: 0.5718 -  
f1_score: 0.5387 - false_negatives: 2255570.0000 - false_positives:  
11804995.0000 - iou_soft: 0.2744 - loss: 0.6093 - true_negatives:  
19572816.0000 - true_positives: 8309660.0000 - val_dice_loss: 0.6353 -  
val_f1_score: 0.1559 - val_false_negatives: 1023029.0000 -  
val_false_positives: 98778.0000 - val_iou_soft: 0.2242 - val_loss:  
0.5158 - val_true_negatives: 4017557.0000 - val_true_positives:  
103516.0000
```

Epoch 2/5

```
40/40 ————— 25s 547ms/step - dice_loss: 0.5198 -  
f1_score: 0.6306 - false_negatives: 1635085.0000 - false_positives:  
8606010.0000 - iou_soft: 0.3182 - loss: 0.5372 - true_negatives:  
22771956.0000 - true_positives: 8929992.0000 - val_dice_loss: 0.5954 -  
val_f1_score: 0.4690 - val_false_negatives: 685729.0000 -  
val_false_positives: 308758.0000 - val_iou_soft: 0.2556 - val_loss:  
0.4926 - val_true_negatives: 3807577.0000 - val_true_positives:  
440816.0000
```

Epoch 3/5

```
40/40 ————— 25s 542ms/step - dice_loss: 0.4936 -  
f1_score: 0.6459 - false_negatives: 1621228.0000 - false_positives:  
7961335.0000 - iou_soft: 0.3416 - loss: 0.5032 - true_negatives:  
23416452.0000 - true_positives: 8944025.0000 - val_dice_loss: 0.5667 -  
val_f1_score: 0.5819 - val_false_negatives: 457768.0000 -  
val_false_positives: 497145.0000 - val_iou_soft: 0.2791 - val_loss:  
0.4824 - val_true_negatives: 3619190.0000 - val_true_positives:  
668777.0000
```

Epoch 4/5

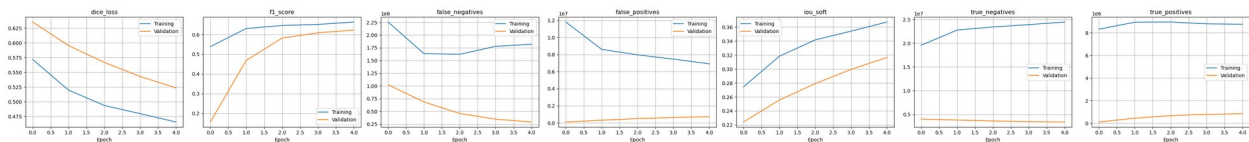
```
40/40 ————— 25s 548ms/step - dice_loss: 0.4797 -  
f1_score: 0.6513 - false_negatives: 1778629.0000 - false_positives:  
7461743.0000 - iou_soft: 0.3541 - loss: 0.4844 - true_negatives:  
23916116.0000 - true_positives: 8786551.0000 - val_dice_loss: 0.5428 -  
val_f1_score: 0.6088 - val_false_negatives: 346592.0000 -  
val_false_positives: 634603.0000 - val_iou_soft: 0.2995 - val_loss:  
0.4742 - val_true_negatives: 3481732.0000 - val_true_positives:  
779953.0000
```

Epoch 5/5

```
40/40 ————— 25s 548ms/step - dice_loss: 0.4656 -  
f1_score: 0.6635 - false_negatives: 1817848.0000 - false_positives:
```

```
6916384.0000 - iou_soft: 0.3671 - loss: 0.4654 - true_negatives:
24461682.0000 - true_positives: 8747122.0000 - val_dice_loss: 0.5235 -
val_f1_score: 0.6225 - val_false_negatives: 290703.0000 -
val_false_positives: 720029.0000 - val_iou_soft: 0.3165 - val_loss:
0.4678 - val_true_negatives: 3396306.0000 - val_true_positives:
835842.0000
```

```
ut.plot_training_history(history)
```



Task 1.3: Visualize model predictions

Make a plot that illustrates the original image, the predicted mask, and the ground truth mask. You can use `visualize_predictions()` in `utilities.py`

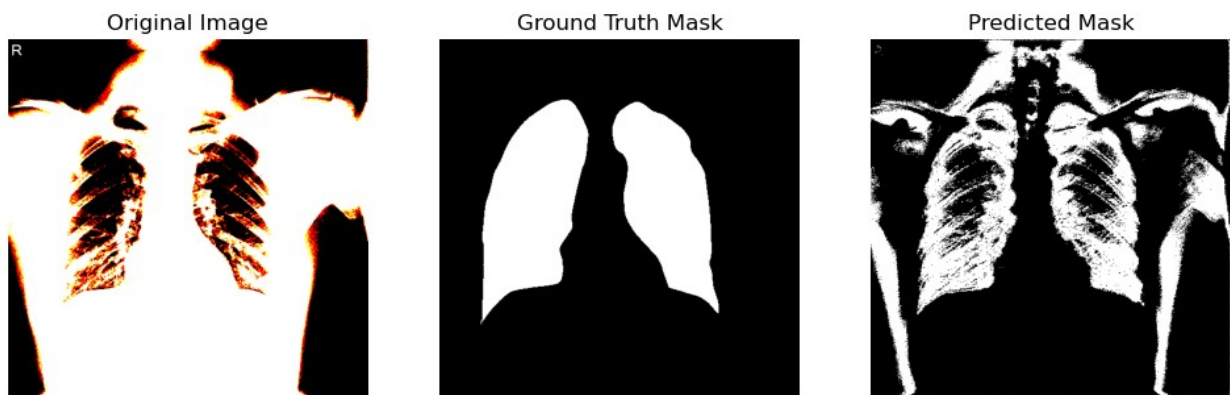
```
test_ds = ut.make_dataset(test, augment_data=False)
test_predict = model.predict(test_ds)
```

```
5/5 ————— 1s 120ms/step
```

```
ut.visualize_predictions(model, test_ds, 3)
```

```
1/1 ————— 1s 650ms/step
```

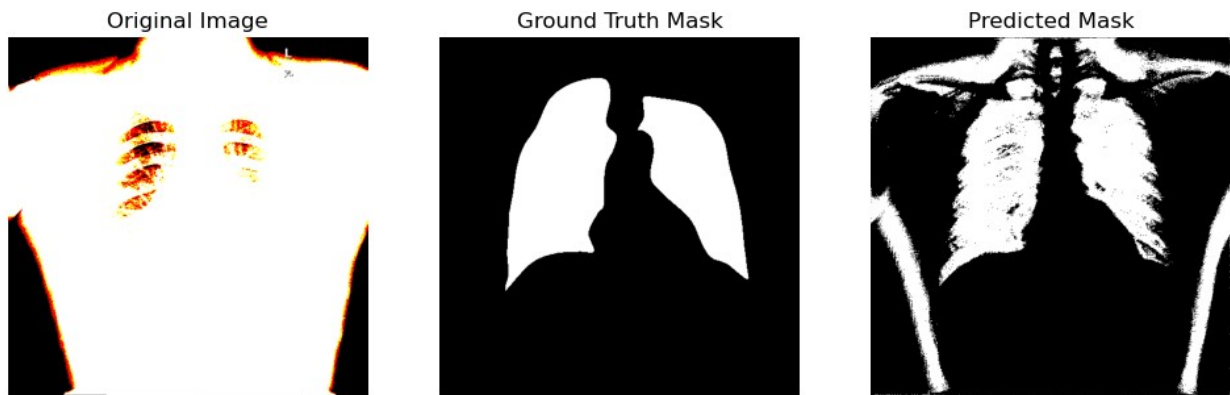
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-58.34..76.0305].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



```
test_true = []
for _, labels_batch in test_ds:
    test_true.extend(labels_batch.numpy())

test_iou = ut.iou_soft(test_true, test_predict)
test_f1 = ut.f1_score(test_true, test_predict)

print("IOU test: ", test_iou.numpy())
print("f1 test: ", test_f1.numpy())

IOU test:  0.25938886
f1 test:  0.49186113
```

Task 2: Implementing U-Net with Transfer Learning

Implement a model with the U-net structure that you have learned about in the lectures, but now with a pre-trained backbone. There are many pre-trained back-bones to choose from. Pick freely from the selection here [tf.keras.applications](#), or here [Keras model scores](#) (nicer table in the second link). Feel free to experiment with the number of layers, loss-function, batch-

normalization, etc. Many of the backbones available are quite big, so you might find it quite time-consuming to train them on your personal computers.

For those with a dedicated graphics card (NVIDIA and AMD) Tensorflow or PyTorch (not syllabus)

And wants to experiment with their own compute resources (can be a lot of fun)

Tensorflow: <https://learn.microsoft.com/en-us/windows/ai/directml/gpu-tensorflow-plugin>

PyTorch: <https://learn.microsoft.com/en-us/windows/ai/directml/pytorch-windows>

Task 2.1: Transfer Learning Model Implementation

In this task, we will implement a U-Net model that uses the pre-trained weights of popular public networks (e.g., **VGG16** and **ResNet50**) as encoders. This is a common transfer learning strategy for segmentation.

Example 1: U-Net with VGG16 backbone

```
from tensorflow.keras.applications import VGG16
```

```
from tensorflow.keras.applications import ResNet50
```

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.layers import Flatten, Dense

conv_base = VGG16(weights='imagenet',
                    include_top=False,
                    input_shape=(512, 512, 3))

for layer in conv_base.layers:
    layer.trainable = False
conv_base.summary()
```

Model: "vgg16"

Layer (type) Param #	Output Shape
input_layer_1 (InputLayer) 0	(None, 512, 512, 3)
block1_conv1 (Conv2D) 1,792	(None, 512, 512, 64)

36,928	block1_conv2 (Conv2D)	(None, 512, 512, 64)	
0	block1_pool (MaxPooling2D)	(None, 256, 256, 64)	
73,856	block2_conv1 (Conv2D)	(None, 256, 256, 128)	
147,584	block2_conv2 (Conv2D)	(None, 256, 256, 128)	
0	block2_pool (MaxPooling2D)	(None, 128, 128, 128)	
295,168	block3_conv1 (Conv2D)	(None, 128, 128, 256)	
590,080	block3_conv2 (Conv2D)	(None, 128, 128, 256)	
590,080	block3_conv3 (Conv2D)	(None, 128, 128, 256)	
0	block3_pool (MaxPooling2D)	(None, 64, 64, 256)	
1,180,160	block4_conv1 (Conv2D)	(None, 64, 64, 512)	
2,359,808	block4_conv2 (Conv2D)	(None, 64, 64, 512)	
2,359,808	block4_conv3 (Conv2D)	(None, 64, 64, 512)	
	block4_pool (MaxPooling2D)	(None, 32, 32, 512)	

0				
	block5_conv1 (Conv2D)		(None, 32, 32, 512)	
2,359,808				
	block5_conv2 (Conv2D)		(None, 32, 32, 512)	
2,359,808				
	block5_conv3 (Conv2D)		(None, 32, 32, 512)	
2,359,808				
	block5_pool (MaxPooling2D)		(None, 16, 16, 512)	
0				

Total params: 14,714,688 (56.13 MB)

Trainable params: 0 (0.00 B)

Non-trainable params: 14,714,688 (56.13 MB)

```
c1 = conv_base.get_layer('block1_conv2').output
c2 = conv_base.get_layer('block2_conv2').output
c3 = conv_base.get_layer('block3_conv3').output
c4 = conv_base.get_layer('block4_conv3').output
c5 = conv_base.get_layer('block5_conv3').output
```

```
"""
print("c1 shape", np.shape(c1))
print("c2 shape", np.shape(c2))
print("c3 shape", np.shape(c3))
print("c4 shape", np.shape(c4))
print("c5 shape", np.shape(c5))
""";
```

```
base_out = conv_base.output
```

```
u6 = Conv2DTranspose(8, (3, 3), strides = (2, 2), padding = "same")
(base_out)
u6 = concatenate([u6, c5])
c6 = conv2d_block(u6, 8, kernel_size = 3, batchnorm = True)

u7 = Conv2DTranspose(8, (3, 3), strides = (2, 2), padding = "same")
(c6)
```

```

u7 = concatenate([u7, c4])
c7 = conv2d_block(u7, 8, kernel_size = 3, batchnorm = True)

u8 = Conv2DTranspose(8, (3, 3), strides = (2, 2), padding = "same")
(c7)
u8 = concatenate([u8, c3])
c8 = conv2d_block(u8, 8, kernel_size = 3, batchnorm = True)

u9 = Conv2DTranspose(8, (3, 3), strides = (2, 2), padding = "same")
(c8)
u9 = concatenate([u9, c2])
c9 = conv2d_block(u9, 8, kernel_size = 3, batchnorm = True)

u10 = Conv2DTranspose(8, (3, 3), strides = (2, 2), padding = "same")
(c9)
u10 = concatenate([u10, c1])
c10 = conv2d_block(u10, 8, kernel_size = 3, batchnorm = True)

"""
print("c6 shape", np.shape(c6))
print("c7 shape", np.shape(c7))
print("c8 shape", np.shape(c8))
print("c9 shape", np.shape(c9))
print("c10 shape", np.shape(c10))

print("u6 shape", np.shape(u6))
print("u7 shape", np.shape(u7))
print("u8 shape", np.shape(u8))
print("u9 shape", np.shape(u9))
print("u10 shape", np.shape(u10))
""";

outputs = Conv2D(1, (1, 1), activation='sigmoid')(c10)

vgg16_model = Model(conv_base.input, outputs)

vgg16_model.compile(
    optimizer="SGD",
    loss=ut.bce_dice_loss,
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        ut.f1_score,
        #accuracy_score,
        ut.iou_soft,
        ut.dice_loss
        # Other metrics of your choice

```

```

    ]
)
vgg16_model.summary()
Model: "functional_1"

```

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 512, 512, 3)	0	-
block1_conv1 input_layer_1[0]... (Conv2D)	(None, 512, 512, 64)	1,792	
block1_conv2 block1_conv1[0][...] (Conv2D)	(None, 512, 512, 64)	36,928	
block1_pool block1_conv2[0][...] (MaxPooling2D)	(None, 256, 256, 64)	0	
block2_conv1 block1_pool[0][0] (Conv2D)	(None, 256, 256, 128)	73,856	
block2_conv2 block2_conv1[0][...] (Conv2D)	(None, 256, 256, 128)	147,584	
block2_pool	(None, 128, 128, 128)	0	

block2_conv2[0][...	(MaxPooling2D)	128)		
block3_conv1	(None, 128, 128,	295,168		
block2_pool[0][0]	(Conv2D)	256)		
block3_conv2	(None, 128, 128,	590,080		
block3_conv1[0][...	(Conv2D)	256)		
block3_conv3	(None, 128, 128,	590,080		
block3_conv2[0][...	(Conv2D)	256)		
block3_pool	(None, 64, 64,	0		
block3_conv3[0][...	(MaxPooling2D)	256)		
block4_conv1	(None, 64, 64,	1,180,160		
block3_pool[0][0]	(Conv2D)	512)		
block4_conv2	(None, 64, 64,	2,359,808		
block4_conv1[0][...	(Conv2D)	512)		
block4_conv3	(None, 64, 64,	2,359,808		
block4_conv2[0][...	(Conv2D)	512)		
block4_pool	(None, 32, 32,	0		
block4_conv3[0][...				

(MaxPooling2D)	512)		
block5_conv1	(None, 32, 32,	2,359,808	
block4_pool[0][0]	(Conv2D)	512)	
block5_conv2	(None, 32, 32,	2,359,808	
block5_conv1[0][...]	(Conv2D)	512)	
block5_conv3	(None, 32, 32,	2,359,808	
block5_conv2[0][...]	(Conv2D)	512)	
block5_pool	(None, 16, 16,	0	
block5_conv3[0][...]	(MaxPooling2D)	512)	
conv2d_transpose_4	(None, 32, 32, 8)	36,872	
block5_pool[0][0]	(Conv2DTranspose)		
concatenate_4	(None, 32, 32,	0	
conv2d_transpose...	(Concatenate)	520)	
block5_conv3[0][...]			
conv2d_19 (Conv2D)	(None, 32, 32, 8)	37,448	
concatenate_4[0]...			
batch_normalizatio...	(None, 32, 32, 8)	32	conv2d_19[0]
[0]	(BatchNormalizatio...		

activation_18 batch_normalizat...	(None, 32, 32, 8)	0	
(Activation)			
conv2d_20 (Conv2D) activation_18[0]...	(None, 32, 32, 8)	584	
batch_normalizatio... [0]	(None, 32, 32, 8)	32	conv2d_20[0]
(BatchNormalizatio...			
activation_19 batch_normalizat...	(None, 32, 32, 8)	0	
(Activation)			
conv2d_transpose_5 activation_19[0]...	(None, 64, 64, 8)	584	
(Conv2DTranspose)			
concatenate_5 conv2d_transpose... (Concatenate)	(None, 64, 64, 520)	0	
block4_conv3[0][...			
conv2d_21 (Conv2D) concatenate_5[0]...	(None, 64, 64, 8)	37,448	
batch_normalizatio... [0]	(None, 64, 64, 8)	32	conv2d_21[0]
(BatchNormalizatio...			
activation_20 batch_normalizat...	(None, 64, 64, 8)	0	
(Activation)			

conv2d_22 (Conv2D)	(None, 64, 64, 8)	584	
activation_20[0]...			
batch_normalizatio...	(None, 64, 64, 8)	32	conv2d_22[0]
[0]			
(BatchNormalizatio...			
activation_21	(None, 64, 64, 8)	0	
batch_normalizat...			
(Activation)			
conv2d_transpose_6	(None, 128, 128,	584	
activation_21[0]...			
(Conv2DTranspose)	8)		
concatenate_6	(None, 128, 128,	0	
conv2d_transpose...			
(Concatenate)	264)		
block3_conv3[0][...			
conv2d_23 (Conv2D)	(None, 128, 128,	19,016	
concatenate_6[0]...			
	8)		
batch_normalizatio...	(None, 128, 128,	32	conv2d_23[0]
[0]			
(BatchNormalizatio...	8)		
activation_22	(None, 128, 128,	0	
batch_normalizat...			
(Activation)	8)		
conv2d_24 (Conv2D)	(None, 128, 128,	584	

activation_22[0]...		8)		
batch_normalization_22[0]	(None, 128, 128,		32	conv2d_24[0]
(BatchNormalization)	8)			
activation_23	(None, 128, 128,		0	
batch_normalization_23	(Activation)	8)		
conv2d_transpose_7	(None, 256, 256,		584	
activation_23[0]...	(Conv2DTranspose)	8)		
concatenate_7	(None, 256, 256,		0	
conv2d_transpose_7	(Concatenate)	136)		
block2_conv2[0]...				
conv2d_25 (Conv2D)	(None, 256, 256,		9,800	
concatenate_7[0]...		8)		
batch_normalization_25[0]	(None, 256, 256,		32	conv2d_25[0]
(BatchNormalization)	8)			
activation_24	(None, 256, 256,		0	
batch_normalization_24	(Activation)	8)		
conv2d_26 (Conv2D)	(None, 256, 256,		584	
activation_24[0]...				

	8)		
batch_normalization_25[0]	(None, 256, 256, 32)	conv2d_26[0]	
(BatchNormalization)	8)		
activation_25	(None, 256, 256, 0)		
batch_normalization_25[0]	(Activation)	8)	
conv2d_transpose_8	(None, 512, 512, 584)		
activation_25[0]	(Conv2DTranspose)	8)	
concatenate_8	(None, 512, 512, 0)		
conv2d_transpose_8	(Concatenate)	72)	
block1_conv2[0]			
conv2d_27 (Conv2D)	(None, 512, 512, 5,192)		
concatenate_8[0]	8)		
batch_normalization_26[0]	(None, 512, 512, 32)	conv2d_27[0]	
(BatchNormalization)	8)		
activation_26	(None, 512, 512, 0)		
batch_normalization_26[0]	(Activation)	8)	
conv2d_28 (Conv2D)	(None, 512, 512, 584)		
activation_26[0]	8)		

batch_normalization_28[0]	(None, 512, 512, 32)	conv2d_28[0]	
(BatchNormalization)	8)		
activation_27	(None, 512, 512, 0)		
batch_normalization_27	(Activation)	8)	
conv2d_29 (Conv2D)	(None, 512, 512, 9)		
activation_27[0]	1)		

Total params: 14,866,049 (56.71 MB)

Trainable params: 151,201 (590.63 KB)

Non-trainable params: 14,714,848 (56.13 MB)

Task 2.2: Train the transfer learning model and plot the training history

Feel free to use the `plot_training_history` function from the provided `library_utilities.py`

```
training_history_vgg16 = vgg16_model.fit(
    train_aug_ds,
    validation_data=val_ds,
    epochs=5,
    batch_size=40,
    shuffle=True,
    verbose=1)

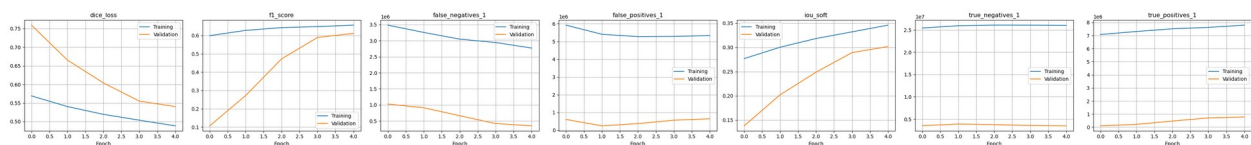
ut.plot_training_history(training_history_vgg16)
```

Epoch 1/5
40/40 ————— 78s 2s/step - dice_loss: 0.5690 - f1_score: 0.5993 - false_negatives_1: 3483835.0000 - false_positives_1: 5923080.0000 - iou_soft: 0.2768 - loss: 0.5717 - true_negatives_1: 25455118.0000 - true_positives_1: 7081013.0000 - val_dice_loss: 0.7593 - val_f1_score: 0.1066 - val_false_negatives_1: 1026627.0000 -

```

val_false_positives_1: 604808.0000 - val_iou_soft: 0.1380 - val_loss:
0.7308 - val_true_negatives_1: 3511527.0000 - val_true_positives_1:
99918.0000
Epoch 2/5
40/40 ————— 73s 2s/step - dice_loss: 0.5404 - f1_score:
0.6279 - false_negatives_1: 3256869.0000 - false_positives_1:
5406661.0000 - iou_soft: 0.3003 - loss: 0.5351 - true_negatives_1:
25971148.0000 - true_positives_1: 7308362.0000 - val_dice_loss: 0.6654
- val_f1_score: 0.2718 - val_false_negatives_1: 911594.0000 -
val_false_positives_1: 242762.0000 - val_iou_soft: 0.2021 - val_loss:
0.5861 - val_true_negatives_1: 3873573.0000 - val_true_positives_1:
214951.0000
Epoch 3/5
40/40 ————— 76s 2s/step - dice_loss: 0.5194 - f1_score:
0.6432 - false_negatives_1: 3048804.0000 - false_positives_1:
5277052.0000 - iou_soft: 0.3182 - loss: 0.5080 - true_negatives_1:
26101012.0000 - true_positives_1: 7516167.0000 - val_dice_loss: 0.6040
- val_f1_score: 0.4724 - val_false_negatives_1: 663132.0000 -
val_false_positives_1: 369045.0000 - val_iou_soft: 0.2488 - val_loss:
0.5264 - val_true_negatives_1: 3747290.0000 - val_true_positives_1:
463413.0000
Epoch 4/5
40/40 ————— 75s 2s/step - dice_loss: 0.5037 - f1_score:
0.6486 - false_negatives_1: 2942452.0000 - false_positives_1:
5288561.0000 - iou_soft: 0.3320 - loss: 0.4889 - true_negatives_1:
26089320.0000 - true_positives_1: 7622712.0000 - val_dice_loss: 0.5550
- val_f1_score: 0.5895 - val_false_negatives_1: 419488.0000 -
val_false_positives_1: 560516.0000 - val_iou_soft: 0.2892 - val_loss:
0.4972 - val_true_negatives_1: 3555819.0000 - val_true_positives_1:
707057.0000
Epoch 5/5
40/40 ————— 75s 2s/step - dice_loss: 0.4885 - f1_score:
0.6567 - false_negatives_1: 2770699.0000 - false_positives_1:
5333934.0000 - iou_soft: 0.3458 - loss: 0.4697 - true_negatives_1:
26043968.0000 - true_positives_1: 7794436.0000 - val_dice_loss: 0.5403
- val_f1_score: 0.6117 - val_false_negatives_1: 348433.0000 -
val_false_positives_1: 642175.0000 - val_iou_soft: 0.3017 - val_loss:
0.4978 - val_true_negatives_1: 3474160.0000 - val_true_positives_1:
778112.0000

```



```

ut.visualize_predictions(vgg16_model, test_ds, 3)

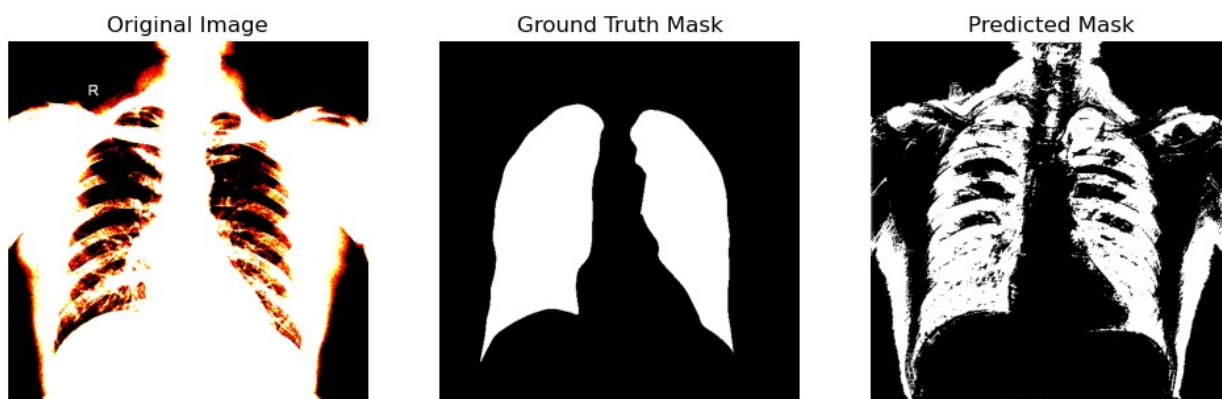
```

```

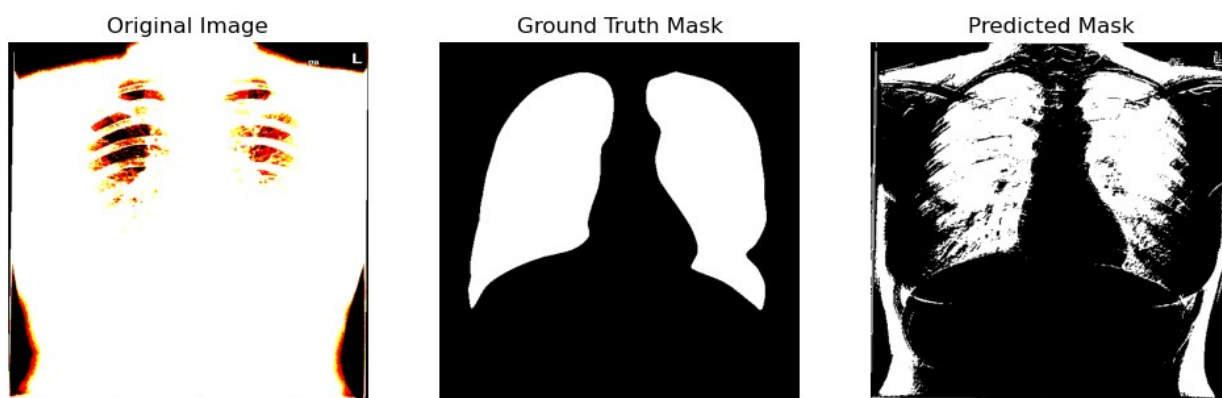
1/1 ————— 2s 2s/step

```

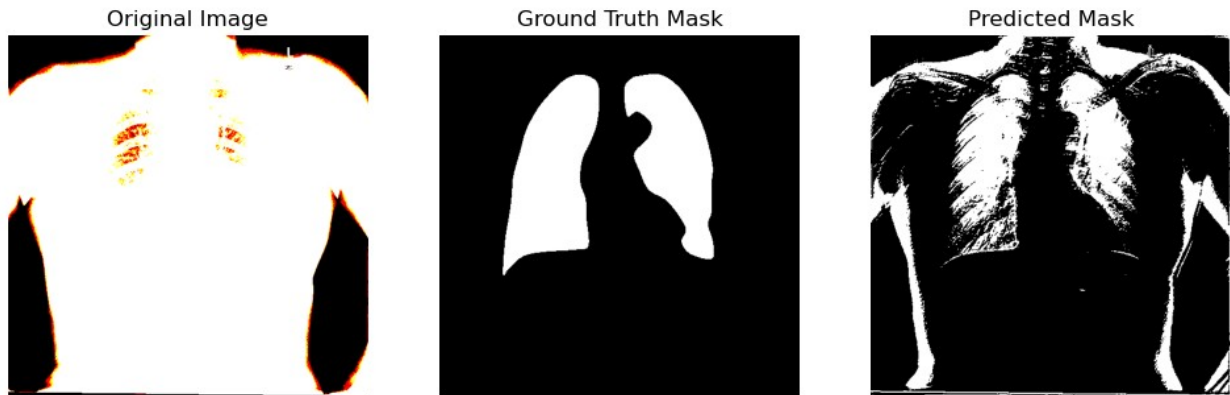
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



```
test_predict_VGG16 = vgg16_model.predict(test_ds)

test_iou_VGG16 = ut.iou_soft(test_true, test_predict_VGG16)
test_f1_VGG16 = ut.f1_score(test_true, test_predict_VGG16)

print("IOU test VGG16: ", test_iou_VGG16.numpy())
print("f1 test VGG16: ", test_f1_VGG16.numpy())

5/5 _____ 7s 1s/step
IOU test VGG16:  0.2643171
f1 test VGG16:  0.5051491
```

Part 2 - Discussion

Question 1: Which model architectures did you explore, and what type of hyperparameter optimization did you try?

Answer 1:

We explored VGG16 as the backbone in the transfer learning. We did not use hyperparameter optimization for this assignment.

Question 2: Which of the models did you choose to train as your final submission, and how long did it take to train?

Answer 2:

For the final submission we trained the model with vgg16 downscaling. The training took about 6 minutes (locally).

Question 3: What were the biggest challenges with this assignment?

Answer 3:

The hardest part of the assignment was the dimensional matching between the convolution layers.